WEEK 10

TITLE

Student Registration and CRUD Operations using React, Express.js and MongoDB

AIM

To develop a Student Registration application using React and Express.js with MongoDB, and perform CRUD operations such as Create, Read, Update and Delete on student records.

ALGORITHM

1. Create a React application containing a Student Registration form. 2. Create form fields for Name, Email, Password, Gender, Country and Languages. 3. Use React **useState** to store and manage the form data.
4. Use **useEffect** to retrieve student records when the page loads. 5. Send a **POST** request to Express.js to create a new student record. 6. Send a **GET** request to retrieve and display student records.
7. Send a **PUT** request to update an existing student record.
8. Send a **DELETE** request to delete a student record.
9. Use Mongoose to connect the Express.js server with MongoDB. 10.Display the student records in a table with Edit and Delete options.

PROGRAM

Student Registration and CRUD Operations using React, Express.js and MongoDB

PROGRAM CODE

App.jsx

```
import './App.css';
import { useEffect, useState } from "react";


const API = "http://localhost:5000";


function App() {


  const emptyForm = {
    name: "",
    email: "",
    password: "",
    gender: "",
    country: "",
    languages: []
  };
```

```
const [form, setForm] = useState(emptyForm); const
[students, setStudents] = useState([]); const [message,
setMessage] = useState(""); const [editId, setEditId] =
useState("");



useEffect(() => {
  loadStudents();
}, []);



// READ
function loadStudents() {


  fetch(API + "/students")
    .then(response => response.json())
    .then(data => {
      setStudents(data);
    })
    .catch(error => {
      console.log(error);
      setMessage("Start the Express server first"); });
}



// Handle input fields
function handleChange(event) {
  const { name, value } = event.target;


  setForm({
    ...form,
    [name]: value
  });
}



// Handle languages
function handleLanguage(event) {
```

```
  const value = event.target.value;
  const checked = event.target.checked;

  let languages = form.languages;

  if (checked) {
    languages = [...languages, value];
  } else {
    languages = languages.filter(item => item !== value); }

  setForm({
    ...form,
    languages: languages
  });
}



// CREATE / UPDATE
function saveStudent(event) {

  event.preventDefault();
  let url = API + "/students";
  let method = "POST";

  if (editId !== "") {
    url = API + "/students/" + editId; method
    = "PUT";
  }

  fetch(url, {
    method: method,

    headers: {
      "Content-Type": "application/json" },

    body: JSON.stringify(form)
  })
```

```
    .then(response => response.json())
    .then(data => {

      setMessage(data.message);
      setForm(emptyForm);
      setEditId("");

      loadStudents();
    });
}



// EDIT
function editStudent(student) {

  setForm({
    name: student.name,
    email: student.email,
    password: student.password,
    gender: student.gender,
    country: student.country,
    languages: student.languages
  });

  setEditId(student._id);
  setMessage("Edit the data and click Update"); }



// DELETE
function deleteStudent(id) {

  fetch(API + "/students/" + id, {
    method: "DELETE"
  })
    .then(response => response.json())
    .then(data => {

      setMessage(data.message);
      loadStudents();
```

```
    });
}


// RESET
function resetForm() {

  setForm(emptyForm);
  setEditId("");
  setMessage("");
}
return (

  <div className="page">

    <div className="form-box"> <h1>Student

      Registration</h1> <form

      onSubmit={saveStudent}>

      <label>Name</label>

        <input
          name="name"
          value={form.name}
          onChange={handleChange}
          required
        />



        <label>Email</label>

        <input
          type="email"
          name="email"
          value={form.email}
          onChange={handleChange}
          required
```

```
/>

<label>Password</label>

<input
  type="password"
  name="password"
  value={form.password}
  onChange={handleChange}
  required
/>

<label>Gender</label>

<div className="row">

  <label>
    <input
      type="radio"
      name="gender"
      value="Male"
      checked={form.gender === "Male"}
      onChange={handleChange}
    />
    Male
  </label>

  <label>
    <input
      type="radio"
      name="gender"
      value="Female"
      checked={form.gender === "Female"}
      onChange={handleChange}
    />
    Female
  </label>
```

```
</div>
<label>Country</label>

<select
  name="country"
  value={form.country}
  onChange={handleChange}
  required
>
  <option value="">Select Country</option> <option
  value="India">India</option> <option
  value="USA">USA</option> <option
  value="UK">UK</option>
  <option value="Australia">Australia</option> </select>

<label>Languages</label>

<div className="row">

  <label>
    <input
      type="checkbox"
      value="English"
      checked={form.languages.includes("English")}
      onChange={handleLanguage}
    />
    English
  </label>

  <label>
    <input
      type="checkbox"
      value="Hindi"
      checked={form.languages.includes("Hindi")}
      onChange={handleLanguage}
    />
    Hindi
  </label>
```

```
      <label>
        <input
          type="checkbox"
          value="Telugu"
          checked={form.languages.includes("Telugu")}
          onChange={handleLanguage}
        />
        Telugu
      </label>

    </div>


    <button type="submit">
      {editId === "" ? "Register" : "Update"} </button>

    <button
      type="button"
      onClick={resetForm}
    >
      Reset
    </button>

  </form>


   {message && (
    <div className="message">
      {message}
    </div>
   )}

</div>


<div className="data-box">
```

```
<h2>Registered Students</h2> <table>

  <thead>

    <tr>
      <th>Name</th>
      <th>Email</th>
      <th>Gender</th>
      <th>Country</th>
      <th>Languages</th>
      <th>Actions</th>
    </tr>

  </thead>



  <tbody>

      {students.map(student => ( <tr

            key={student._id}>

        <td>{student.name}</td>
        <td>{student.email}</td>
        <td>{student.gender}</td>
        <td>{student.country}</td>
        <td>{student.languages.join(", ")}</td>

        <td>

          <button
            onClick={() => editStudent(student)}
          >
            Edit
          </button>

          <button
            onClick={() => deleteStudent(student._id)} >
            Delete
```

```
                    </button>

                  </td>

                </tr>

              ))}

            </tbody>

          </table>

        </div>

      </div>
    );
}
export default App;
```

server.js

```
import express from "express";
import mongoose from "mongoose";
import cors from "cors";

const app = express();

app.use(cors());
app.use(express.json());



// MongoDB connection
mongoose
  .connect("mongodb://127.0.0.1:27017/studentdb") .then(() => {
    console.log("MongoDB connected");
  })
  .catch((error) => {
    console.log("MongoDB connection error:", error.message); });
```

```
// Schema
const studentSchema = new mongoose.Schema({ name:
  String,
  email: String,
  password: String,
  gender: String,
  country: String,
  languages: [String]
});
// Model
const Student = mongoose.model("Student", studentSchema);



// Test route
app.get("/", (req, res) => {
  res.send("Express server is working");
});



// READ
app.get("/students", async (req, res) => {

  const students = await Student.find();

  res.json(students);
});



// CREATE
app.post("/students", async (req, res) => {

  const student = new Student(req.body);

  await student.save();

  res.json({
```

```
    message: "Registration successful"
  });
});



// UPDATE
app.put("/students/:id", async (req, res) => {
  await Student.findByIdAndUpdate(
    req.params.id,
    req.body
  );

  res.json({
    message: "Student updated successfully" });
});



// DELETE
app.delete("/students/:id", async (req, res) => { await

  Student.findByIdAndDelete(req.params.id);

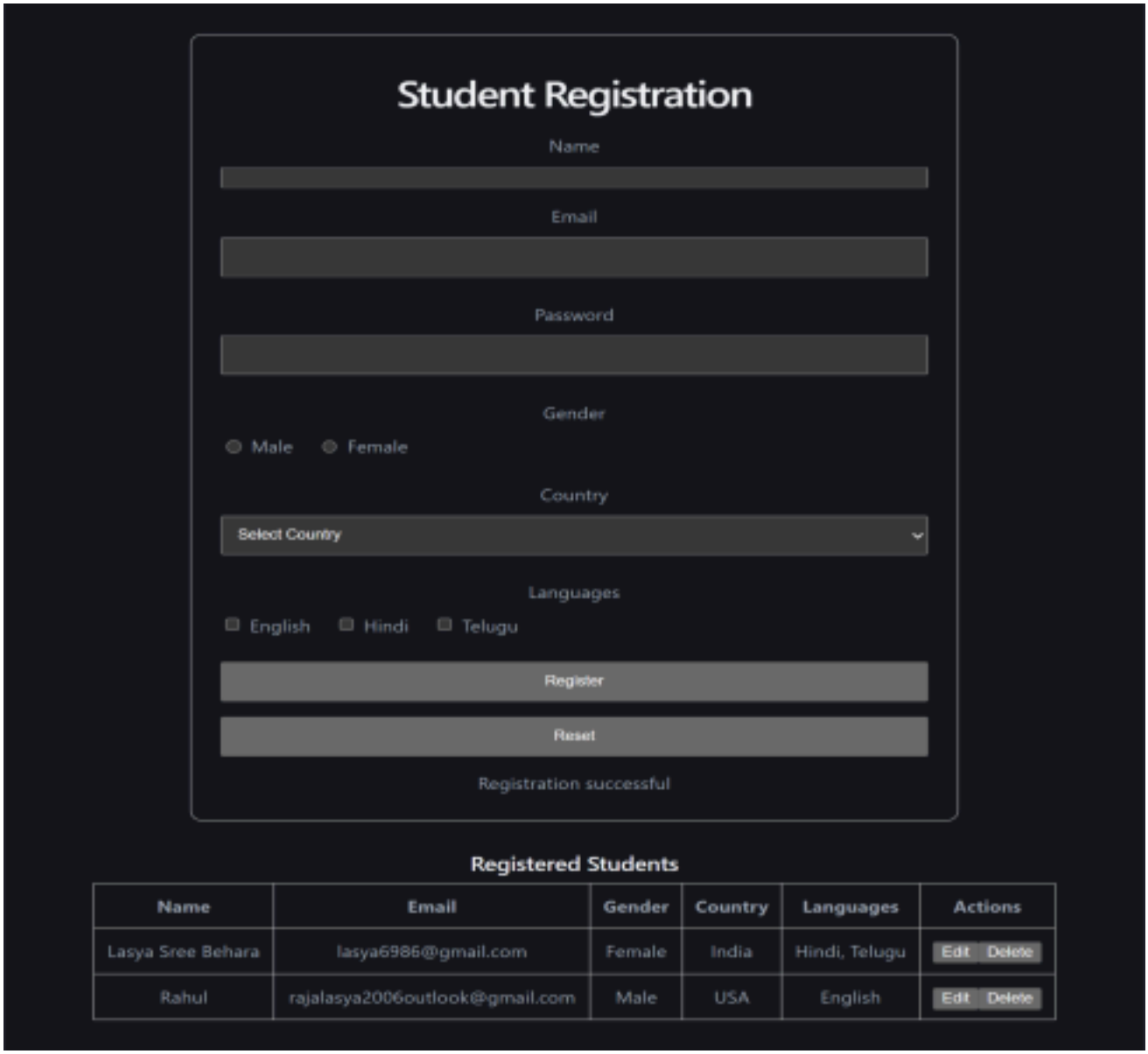
  res.json({
    message: "Student deleted successfully" });
});



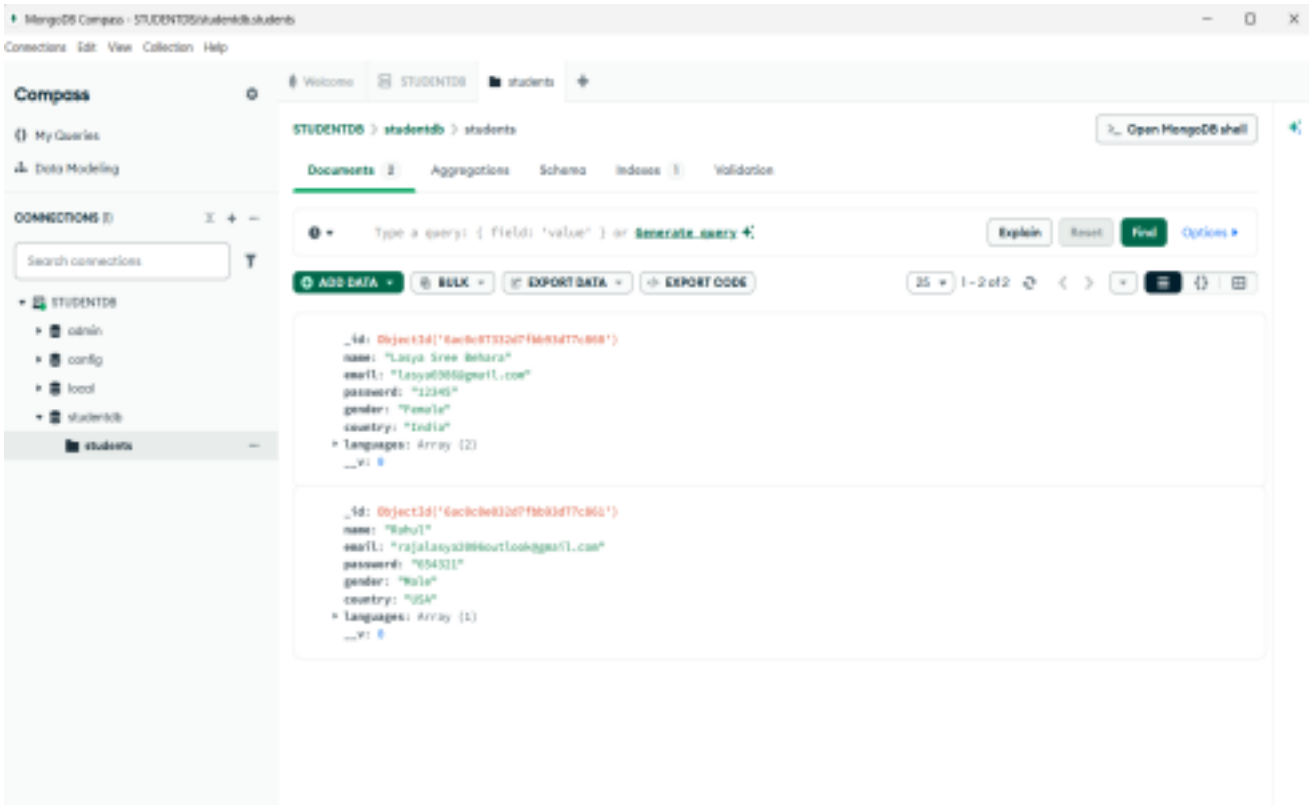
// Start server
app.listen(5000, () => {
  console.log("Server running on port 5000"); });
```

OUTPUTS:

Output 1 – Filled Student Registration Form



Output 2 – Student Records in MongoDB



RESULT

The Student Registration application was successfully developed using **React, Express.js and MongoDB**. The application successfully performs **Create, Read, Update and Delete (CRUD) operations** and displays the student records dynamically.